

🏠 > 📁 Functional Programming, SWU (Spring 2026) > 📖 Assignment 4 🔹

📖 **Assig...** Available since 3 months · Due 2 months ago · Locked 2 months ago · **Grades published**

Assignment description



In this assignment we will start work on the interpreter that you will finish by the end of the course. More precisely we will write preliminary versions of code that

- evaluates arithmetic expressions
- evaluates boolean expressions
- Write a pretty-printer for both boolean and arithmetic expressions

The [code template](#)

- **Language.fs**, which contains a description of the language and error messages (the latter is used for the Yellow assignments)
- **Eval.fs**, which contains the actual interpreter.

Read through **Language.fs** carefully to see what we have given you for free. It includes arithmetic expressions, boolean expressions, and a lot of useful infix operators and examples.

You need to hand in **Eval.fs**

Important: To pass this assignment you have to pass either the **Green** or the **Yellow** tests, but not both (they use different types). We recommend that you start with the **Green** ones and then refactor to use the **Yellow** ones.

Doubly important: Remember that all uses of tools like ChatGPT or CoPilot are strictly prohibited and considered academic misconduct. Write your code yourself.

Green Questions

Question 4.1

Create a function **aexprToString** of type **aexpr -> string** that given an

arithmetic expression **a** returns a string version of **a** where

- all sub-expressions except numbers are parenthesised
- there is a space between the infix operators and the operands

Examples:

```
> aexprToString (Num 4)
- val it: string = "4"

> aexprToString (Num 4 .+. Num 2 .* Num 3)
- val it: string = "(4.+.(2.*3))"
..
> aexprToString ((Num 4 .+. Num 2) .* Num 3)
- val it: string = "((4.+2).*3)"
..
> aexprToString ((Num 18 ./ Num 3) ./ Num 2)
- val it: string = "((18./3)./2)"

> aexprToString (Num 18 ./ (Num 3 ./ Num 2))
- val it: string = "(18./(3./2))"

> aexprToString (Num 42 .* (Num 13 .%. Num 3))
- val it: string = "(42.*(13.+( ((13./3).*3).*-1)))"
```

Question 4.2

Create a function **bexprToString** of type **bexpr -> string** that given an boolean expression **b** returns the a string version of **b** where

- all sub-expressions except TT are parenthesised
- there is a space between the infix operators and the operands and
 - **TT** is translated to **true**
 - **Eq** is translated to **=**
 - **Lt** is translated to **<**
 - **Conj** is translated to **/**
 - **Not** is translated to **not**

Examples:

```

> bexprToString TT
- val it: string = "true"

> bexprToString FF
- val it: string = "(not true)"

> bexprToString (Num 42 . = . Num 32)
- val it: string = "(42 = 32)"

> bexprToString (Num 42 . < . Num 32 . + . Num 10)
- val it: string = "(42 < (32 + 10))"

> bexprToString ((Num 42 . < . Num 32 . + . Num 10) . || . (Num 32 . >= .
Num 27 . % . Num 25))
- val it: string =
  "(not ((not (42 < (32 + 10))) /\ (not (not (32 < (27 + (((27 /
25) * 25) * -1))))))"

```

If these examples look strange to you have a look in [Language.fs](#) to make sure you understand how some infix operators, like `||`, are defined in terms of existing ones. This is to keep the amount of code you have to write to a minimum.

Question 4.3

Create a function `aexprEval` of type `aexpr -> int option` that given an arithmetic expression `a` pattern matches on `a` and, without using `Option.bind`, returns

- `Some x` if `a` is equal to `Num x`
- `Some (x + y)` if `a` is equal to `Add(b, c)` and `b` evaluates to `Some x` and `c` evaluates to `Some y`.
- `Some (x * y)` if `a` is equal to `Mul(b, c)` and `b` evaluates to `Some x` and `c` evaluates to `Some y`.
- `Some (x / y)`, where `/` is integer division, if `a` is equal to `Div(b, c)` and `b` evaluates to `Some x` and `c` evaluates to `Some y`, where `y` is not equal to `0`.
- `None` otherwise

```
> aexprEval (Num 4)
- val it: int option = Some 4

> aexprEval (Num 4 .+. Num 2 .*. Num 3)
- val it: int option = Some 10

> aexprEval ((Num 4 .+. Num 2) .*. Num 3)
- val it: int option = Some 18

> aexprEval ((Num 4 .+. Num 2) ./. Num 0)
- val it: int option = None

> aexprEval (Num 42 .*. (Num 13 .%. Num 3))
- val it: int option = Some 42

> aexprEval (Num 42 .*. (Num 13 .%. Num 0))
- val it: int option = None
```

Question 4.4

Create a function `aexprEval2` that behaves exactly the same as `aexprEval` but which uses `Option.bind`. You may not pattern match on options in any way, such as the ones obtained by using recursive calls to `aexprEval2`, and you may not use `aexprEval`.

Question 4.5

Create a function `bexprEval` of the type `bexpr -> bool option` that given a boolean expression `b` returns, using `aexprEval` or `aexprEval2` where necessary, returns

- `Some true` if `b` is equal to `TT`
- `Some(x = y)` if `b` is equal to `Eq(a, c)` and `a` evaluates to `Some x` and `c` evaluates to `Some y`.
- `Some(x < y)` if `b` is equal to `Lt(a, c)` and `a` evaluates to `Some x` and `c` evaluates to `Some y`.
- `Some(x && y)` if `b` is equal to `Conj(a, c)` and `a` evaluates to `Some x` and `c` evaluates to `Some y`.

- **Some(not x)** if **b** is equal to **Not a** and **a** evaluates to **Some x**.
- **None** otherwise

Examples:

```
> bexprEval TT
- val it: bool option = Some true

> bexprEval FF
- val it: bool option = Some false

> bexprEval (Num 42 .=. Num 32)
- val it: bool option = Some false

> bexprEval (Num 42 .<. Num 32 .+ . Num 10)
- val it: bool option = Some false
..
> bexprEval ((Num 42 .<. Num 32 .+ . Num 10) . || . (Num 32 .>= . Num
27 .%. Num 25))
- val it: bool option = Some true
```

..

Question 4.6

We conclude by writing a better pretty-printer that only prints the parentheses that are strictly required. Our original pretty printer **aexprToString** would turn the expression **Num 4 .+ . Num 2 .* . Num 3** to the string **"(4 + (2 * 3))"**, but the string **"4 + 2 * 3"** is cleaner and not loaded with unnecessary parentheses. We have to be careful though, removing the wrong parentheses will give us a string that does not match the original expression.

To get the correct results we will introduce operator levels where the intuition is that if we have the expressions **e1**, **e2**, and **e3**, and the operators **op1** and **op2** and we want to parenthesise the expression **e1 op1 e2 op2 e3** then the subexpression **e2 op2 e3** will be parenthesised only if **op1** is on a strictly lower level than **op2** (except for if **op1** is division which we will come back to later).

We will use the following levels for expressions

- Numbers live on level 0
- multiplication and division live on level 1
- additions live on level 2

This means that the expressions

- `Num 3 *. (Num 4 .+. Num 5)` becomes `"3 * (4 + 5)"` because multiplication lives on level 1 which is strictly less than addition that lives on level 2.
- `(Num 3 *. Num 4) .+. Num 5` becomes `"3 * 4 + 5"` because addition lives on level 2 which is **not** strictly less than multiplication that lives on level 1.

The operator to watch out for is division, because it is not associative. While we have that `(3 + 4) + 5` is the same as `3 + (4 + 5)`, it is not the case that `(18 / 2) / 3` (which is 3) is the same as `18 / (2 / 3)` which is a division by zero since `/` is integer division.

What this means in practice is if you have the expression `e1 / e2 op e3` then you should parenthesise the sub expression `e2 op e3` if the level of `op` is strictly less than **or equal to** 1 (the level of `/`) and not just strictly less than.

Hint: The easiest solution here is if you create an auxiliary function of type `aexpr -> int * string` that given an expression `a` returns a tuple `(level, str)` where `level` is the level of the topmost operator of `a` and `str` is the string translation of `a`. This allows you to, when you recurse through your expression tree, to compare the levels of the results of your recursive calls with the level you are currently on.

Create a function `aexprToString2` that behaves exactly the same as `aexprToString` but which only includes the parentheses that are strictly necessary.

Examples:

```
> aexprToString2 (Num 4 .+. Num 2 *. Num 3)
- val it: string = "4 + 2 * 3"
```

```

> aexprToString2 ((Num 4 .+. Num 2) .*. Num 3)
- val it: string = "(4.+2)*3"

> aexprToString2 ((Num 18 ././ Num 3) ././ Num 2)
- val it: string = "18./3./2"

> aexprToString2 (Num 18 ././ (Num 3 ././ Num 2))
- val it: string = "18./ (3./2)"

> aexprToString2 (Num 42 .*. (Num 13 .%. Num 3))
- val it: string = "42.*(13.+13./3*3*-1)"

```

Question 4.7

Create a function **bexprToString2** that behaves exactly the same as **bexprToString** but which only includes the parentheses that are strictly necessary.

We will use the following levels for boolean expressions

- TT lives on level 0
- not lives on level 1
- equality and less than lives on level 2
- conjunction lives on level 3

Additionally the **not** operator behaves the same as division for arithmetic expressions so if you have **not e** then **e** is parenthesised if the level of **not**, which is 1, is strictly smaller than or equal to the level of **e**.

Examples:

```

> bexprToString2 TT
- val it: string = "true"

> bexprToString2 FF
- val it: string = "not true"

> bexprToString2 (Num 42 .=. Num 32)
- val it: string = "42.=32"

```

```
> bexprToString2 (Num 42 .<. Num 32 .+. Num 10)
- val it: string = "42.<.32.+10"

> bexprToString2 ((Num 42 .<. Num 32 .+. Num 10) .||. (Num 32 .>=.
  Num 27 .%. Num 25))
- val it: string =
  "not (not (42.<.32.+10) /\ not (not (32.<.27.+27./25*25*-1)))"
```

Yellow Questions

All error handling up to this point has been handled through the **option** type, which either gives you an answer or tells you that something went wrong but you don't know what.

In **Language.fs** you will find the **Error** type that contains possible errors.

Question 4.8

Solve Question 4.1 from the green exercises.

Question 4.9

Solve Question 4.2 from the green exercises

Question 4.10

Refactor **aexprEval** from Question 4.3 so that it returns **DivisionByZero** if the user attempts to divide by zero. In 4.3 you were not allowed to use **Option.bind**, and similarly, you must not use **Result.bind** here.

Question 4.11

Create a function **aexprEval2** that behaves exactly like **aexprEval** but which uses **Result.bind**. You may not pattern match on results in any way, such as the ones obtained by using recursive calls to **aexprEval2**, and you may not use **aexprEval**.

Question 4.12

Refactor `bexprEval` above to return a `Result<bool, Error>` rather than a `bool option`. Boolean expressions cannot generate errors on their own, but you must forward anyones generated by `aexprEval` (or `aexprEval2`).

Question 4.13

Solve Question 4.6 from the green exercises.

Question 4.14

Solve Question 4.7 from the green exercises

Red Questions

All functions that we have created in the green and yellow exercises follow a similar pattern -- recurse over the arithmetic and boolean expressions to compute their value. In this assignment we will create two fold functions, one for arithmetic and one for boolean expressions, that allows us to create the `toString` and `eval` functions from the green and the yellow exercises.

So far we have only done fold functions on lists. The folds in this assignment differ in a few key cases

- There is one function per operator
- The accumulator is not carried around as they are for lists but the initial values are computed at the leaves (`Num` for arithmetic expressions and `TT` for boolean expressions) and then propagated upwards through the recursive chain.

All of this will be made clearer with examples.

Question 4.15

Create a function `aexprFold` of type

```
→ (int -> 'a) -> ..... (* folder for numbers *)
.... ('a -> 'a -> 'a) -> ... (* folder for plus *)
→ ('a -> 'a -> 'a) -> ... (* folder for multiplication *)
.... ('a -> 'a -> 'a) -> ... (* folder for division *)
.... aexpr -> .
.... 'a
```

that given a folding function `fnum` for numbers, `fadd` for additions, `fmul` for multiplicatinos, and `fdiv` for division, and an arithmetic expression `a`, returns

- `fnum v`, if `a` is equal to `Num a`
- `fadd acc1 acc2`, if `a` is equal to `Add(e1, e2)` and folding `e1` computes `acc1` and folding `e2` computes `acc2`
- `fmul acc1 acc2`, if `a` is equal to `Mul(e1, e2)` and folding `e1` computes `acc1` and folding `e2` computes `acc2`
- `fdiv acc1 acc2`, if `a` is equal to `Div(e1, e2)` and folding `e1` computes `acc1` and folding `e2` computes `acc2`

The following example creates a function `simplAEval` that works like `aexprEval` except that it doesn't use options, but just crashes if we try to divide by zero.

Examples:

```
let simpleAEval = aexprFold id (+) (.*.) (./)

> simpleAEval (Num 42)
- val it: int = 42

> simpleAEval (Num 42 ./ Num 2)
- val it: int = 21

simpleAEval (Num 42 ./ Num 0)
!!! Crashes because of division by zero
```

Question 4.16

Create the **non-recursive** functions `aexprToString3`, `aexprEval3`, and `aexprToString4` that behaves like the green (or yellow) functions `aexprToString`, `aexprEval`, and `aexprToString2` respectively. The only recursive function you may call is `aexprFold`. If your folding functions (`fadd` for example) gets too long, then define it as an external (or internal) function.

Hint: For the `aexprToString4` function it helps if your accumulator has the

type `(int * string)` where the integer is the level of the outermost operator of the expression that the string represents.

Question 4.17

Create a function `bexprFold` of type

```
→ (aexpr → 'a) → ... (* Arithmetic expression folder *)
... 'b → ... (* Initial accumulator for true *)
... ('a → 'a → 'b) → (* folder for equality *)
... ('a → 'a → 'b) → (* folder for less than *)
... ('b → 'b → 'b) → (* folder for conjunction *)
... ('b → 'b) → ... (* folder for not *)
... bexpr →
... 'b
```

that given a folding function `faexpr` for arithmetic expressions, an initial value `acc` for `TT`, a folding function `feq` for equality, `flt` for less than, `fconj` for conjunction, `fnot` for not, and a boolean expression `b`, returns

- `acc` if `b` is `TT`
- `feq acc1 acc2` if `b` is equal to `Eq(e1, e2)` and folding `e1` computes `acc1` and folding `e2` computes `acc2`
- `flt acc1 acc2` if `b` is equal to `Lt(e1, e2)` and folding `e1` computes `acc1` and folding `e2` computes `acc2`
- `fconj acc1 acc2` if `b` is equal to `Conj(e1, e2)` and folding `e1` computes `acc1` and folding `e2` computes `acc2`
- `fnot acc1` if `b` is equal to `Not e` and folding `e` computes to `acc1`

Note that the folding function for arithmetic expressions completely handles the arithmetic expressions for the equality and the left-hand cases.

Similarly as for `aexprFold` the following example creates a function `simplBEval` that works like `bexprEval` except that it doesn't use options or results, but just crashes if we try to divide by zero.

Examples:

```
let simpleAEval = aexprFold id (+) (* *) (/)
```

```

let simpleBEval = bexprFold simpleAEval true (=) (<) (&&) not

> simpleBEval TT
- val it: bool = true

> simpleBEval (Num 4 .> . Num 32 .%. Num 2)
- val it: bool = true

> simpleBEval (Num 4 .> . Num 32 .%. Num 0)
!!! Crashes because of division by zero

```

Question 4.18

Create the **non-recursive** functions `bexprToString3`, `bexprEval3`, and `bexprToString4` that behaves like the green (or yellow) functions `bexprToString`, `bexprEval`, and `bexprToString2` respectively. The only recursive function you may call is `bexprFold`. If your folding functions (`fconj` for example) gets too long, then define it as an external (or internal) function.

Hint: For the `bexprToString4` function it helps if your accumulator has the type `(int * string)` where the integer is the level of the outermost operator of the expression that the string represents.